

Numele si prenumele (cu MAJUSCULE): _____ Grupa: _____

Test: _____ Tema: _____ Colocviu: _____ FINAL: _____

Test de laborator - Arhitectura Sistemelor de Calcul

16 ianuarie 2025

Seria 14, Varianta 2

- Nota maxima pe care o puteti obtine este 10.
- Nota obtinuta trebuie sa fie minim 5 pentru a promova, fara nicio rotunjire superioara.
- Orice tentativa de fraudă este considerata o incalcare a Regulamentului de Etica!

1 Partea 0x00: x86 - maxim 6p

Presupunem ca aveti acces la un executabil `exec`, pe care il inspectati cu `objdump -d exec`. In momentul in care rulati aceasta comanda, va opriti asupra urmatorului cod. Analizati-l si raspundeti intrebarilor de mai jos. Pentru fiecare raspuns in parte, veti preciza si instructiunile care v-au ajutat in rezolvare.

```
000011ad <f>:
11b1: 55          push    %ebp
11b2: 89 e5      mov     %esp,%ebp
11b4: 83 ec 10   sub     $0x10,%esp
11bc: 05 20 2e 00 00 add    $0x2e20,%eax
11c1: c7 45 f8 00 00 00 00 movl    $0x0,-0x8(%ebp)
11c8: c7 45 fc 00 00 00 00 movl    $0x0,-0x4(%ebp)
11cf: eb 28      jmp     11f9 <f+0x4c>
11d1: 8b 45 f8   mov     -0x8(%ebp),%eax
11d4: 8d 14 85 00 00 00 00 lea     0x0(,%eax,4),%edx
11db: 8b 45 08   mov     0x8(%ebp),%eax
11de: 01 d0      add     %edx,%eax
11e0: 8b 00      mov     (%eax),%eax
11e2: 8b 55 10   mov     0x10(%ebp),%edx
11e5: 89 d1      mov     %edx,%ecx
11e7: 2b 4d 14   sub     0x14(%ebp),%ecx
11ea: 8b 55 f8   mov     -0x8(%ebp),%edx
11ed: 01 d1      add     %edx,%ecx
11ef: 99         cltd
11f0: f7 f9     idiv    %ecx
11f2: 01 45 fc   add     %eax,-0x4(%ebp)
11f5: c1 65 f8 02 shll    $0x2,-0x8(%ebp)
11f9: 8b 45 f8   mov     -0x8(%ebp),%eax
11fc: 8d 14 85 00 00 00 00 lea     0x0(,%eax,4),%edx
1203: 8b 45 08   mov     0x8(%ebp),%eax
1206: 01 d0      add     %edx,%eax
1208: 8b 00      mov     (%eax),%eax
120a: 85 c0      test    %eax,%eax
120c: 7f c3      jg      11d1 <f+0x24>
120e: 8b 45 fc   mov     -0x4(%ebp),%eax
1212: c3        ret

g00 <g>:
g01:          pushl   %ebp
g02:          movl    %esp, %ebp
g03:          subl    $20, %esp
g04:          movl    $1, -8(%ebp)
g05:          movl    $0, -4(%ebp)
g06:          jmp     .L6
          .L7:
g07:          movl    12(%ebp), %eax
g08:          subl    -4(%ebp), %eax
g09:          movl    -4(%ebp), %edx
g0A:          leal    0(,%edx,4), %ecx
g0B:          movl    8(%ebp), %edx
g0C:          addl    %ecx, %edx
g0D:          pushl   $3
g0E:          pushl   $2
g0F:          pushl   %eax
g10:          pushl   %edx
g11:          call    f
g12:          addl    $16, %esp
g13:          movl    %eax, -20(%ebp)
g14:          fildl   -20(%ebp)
g15:          fmul    16(%ebp)
g16:          fstps   -20(%ebp)
g17:          cvttss2sil -20(%ebp), %eax
g18:          movl    %eax, -12(%ebp)
g19:          movl    -8(%ebp), %eax
g1A:          imull   -12(%ebp), %eax
g1B:          movl    %eax, -8(%ebp)
g1C:          addl    $1, -4(%ebp)
          .L6:
g1D:          movl    -4(%ebp), %eax
g1E:          cmpl    12(%ebp), %eax
g1F:          jl      .L7
g20:          fildl   -8(%ebp)
g21:          ret
```

a. (0.75p) Cate argumente primeste procedura `f` si cum ati identificat acest numar de argumente?

b. (0.75p) Ce tip de date returneaza procedura `f` si cum ati identificat acest tip?

- c. (0.75p) In timp ce analizati executabilul, va ganditi sa testati cu o valoare de tip *float*. Alegeti valoarea **-64.5**. Care este reprezentarea acestei valori pe formatul **single** (32b)? Scrieti valoarea in hexa.
- d. (0.75p) Va atrage atentia codificarea hexa a programului si vreti sa vedeti care este semantica reprezentarilor. In acest caz, vreti sa vedeti cum se reprezinta in codul obiect **-0x4(%ebp)**. Observati ca aparitia este destul de rara, asa ca va bazati pe alte instructiuni pentru a deduce acest lucru. Analizati liniile **11d1**, **11db**, **120e**. Ce concluzie puteti trage?
- e. (0.5p) Procedura **f** contine o structura repetitiva. Identificati toate elementele acestei structuri: initializarea contorului, conditia de a ramane in structura, respectiv pasul de continuare (operatia asupra contorului).
- f. (1p) Analizati acum procedura **g**. Primul lucru pe care il observati sunt instructiunile specifice pentru a lucra cu **floating point**. Identificati **fildl op** care incarca intregul op ca **float** pe stiva FPU (in **%st(0)**) si **fmuls op** care efectueaza pe formatul **float** operatia **%st(0) := %st(0) * op**. Avand aceste informatii, determinati care este structura repetitiva si ce se calculeaza in acea structura.

g. (0.5p) Considerati rescrierea instructiunilor pe stiva FPU din procedura `g` in SIMD. Care este echivalentul lor?

h. (1p) Observati ca la linia `g11` din procedura `g` se face un *call* imbricat in procedura `f`. Reprezentati configuratia stivei de apel, in momentul in care se obtine adancimea maxima.

2 Partea 0x01: RISC-V - maxim 3p

a. (0.75p) Presupunem ca avem urmatorul apel `C` din `main` $f(g(x) + 1)$. In urma unor optimizari, compilatorul o sa foloseasca urmatoarele instructiuni de salt `call f`, respectiv `jmp g`. Se va produce vreodata revenirea in `main` in cazul unui procesor RISC-V? Dar al unui x86 (considerand instructiunea de salt in `g` corespunzatoare `j g`)?

b. (0.75p) Sa presupunem ca lucrati la designul unui nou procesor RISC-V si doriti sa adaugati o extensie proprie. Aceasta extensie va contine printre alte 2 instructiuni noi `instr1` avand formatul I si urmatoarele specificatii (`opcode = 0b0000101` si `funct3 = 0b000`) si `instr2` avand formatul U (`opcode = 0b0000101`). Este aceasta o decizie corecta? Explicati.

c. (0.75p) Ce valoare va fi depozitata in `a0` in urma executiei urmatoarelor instructiuni, stiind ca `pc` este intial 0? Prezentrati efectul fiecarei instructiuni.

```
auipc a0, 0x12345
auipc a1, 0x12345
beq a0, a1, label
slli a0, a1, 8
j final
label:
srli a0, a1, 8
final:
```

- d. (0.75p) Se da urmatorul schelet de functie. Care este efectul sau?

```
proc:
    addi sp, sp, -16
    addi s0, sp, 8
    sw ra, 4(s0)
    sw s0, 0(s0)

    // Cod care nu mai modifica valoarea lui sp

    lw s0, 0(s0)
    lw ra, 4(s0)
    addi sp, sp, 16
    ret
```

3 Partea 0x02: Performanta si cache - maxim 1p

- a. (0.5p) Considerăm un sistem de calcul de 32 de biți. Sistemul poate realiza operațiile următoare: operații aritmetice/logice (1 cicli), operații de citire/scriere date în memorie (2 cicli) și operații de branch/salt (4 cicli). Pentru ca operațiile aritmetice/logice să fie executate programul realizează o pseudoinstructiune compusa din instructiunea aritmetica/logica propriu-zisă, două instrucțiuni de citire (citirea operanzilor) și apoi o operație de scriere (scrierea rezultatului). Avem un program care are în componență 20% pseudoinstructiuni aritmetice/logice, 60% alte operații de citire/scriere (30% operații citire și 30% operații scriere) și 20% operații de branch/salt. Presupunem că adăugăm o nouă instrucțiune pentru a înlocui pseudoinstructiunea aritmetica/logica care include cele două citiri și scrierea rezultatului. Noua instrucțiune are nevoie de 4 cicli. Cât de mult (procentual) este îmbunătățit sistemul de calcul?
- b. (0.5p) Un sistem are o memorie principală de 2^{24} bytes iar cache-ul are o capacitate totală de 16 KB, cu o dimensiune a unui bloc de 32 bytes (atât pentru memoria principală, cât și pentru cache). Calculați numărul total de blocuri din memoria principală. Determinați numărul de linii (blocuri) din cache. În cazul unei scheme de mapare directă, presupunem că avem la linia 20, tag-ul 0b0001001011. Cărei adrese din memoria principală îi corespunde adresa de la offsetul 28 (word-ul cu offsetul 7) de pe această linie?

